

APOLLO INSTITUTE OF COMPUTER APPLICATION

BACHELOR OF COMPUTER APPLICATION

SEMESTER 03

Object Oriented Programming with Java (BC03001021)

Assignment 3

UNIT 3: Building Blocks of the Language

### 1-MARK

#### 1. What is the use of **static**?

The **static** keyword is used to create a **class-level member** that is shared by all objects of the class.

#### 2. What is a constructor?

A constructor is a **special method used to initialize an object**. It has the same name as the class and has no return type.

#### 3. What is a default constructor?

A default constructor is a constructor **without parameters** that initializes an object with default values.

#### 4. What is a parameterized constructor?

A parameterized constructor is a constructor that **accepts one or more parameters** to initialize an object with specific values.

#### 5. What is a copy constructor?

A copy constructor is a constructor that **creates a new object by copying the values of another object**.

#### 6. What is method overloading?

Method overloading means having **multiple methods with the same name but different parameters** in the same class.

#### 7. What is method overriding?

Method overriding occurs when a **subclass provides its own implementation of a method already defined in its superclass**.

## 8. What is the use of **this** keyword?

The **this** keyword is used to **refer to the current object of a class**.

## 9. What is a class?

A class is a **blueprint or template for creating objects**. It contains data members and methods.

## 10. What is an object?

An object is an **instance of a class** that has state and behavior.

## 11. What is constructor overloading?

Constructor overloading means having **multiple constructors in the same class with different parameter lists**.

### 7-MARK

## Q.1. What is constructor? Explain types of constructor?

### Definition

A **constructor** is a special member of a class that is used to **initialize objects**.

- The constructor has the **same name as the class**.
- It does **not have a return type**, not even **void**.
- It is automatically called when an object is created.
- A class can have more than one constructor.

### Syntax

```
class ClassName {  
    ClassName() {  
        // initialization  
    }  
}
```

### Example

```
class Student {  
    int rollNo;  
  
    Student() {  
        rollNo = 10;  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        System.out.println(s.rollNo);  
    }  
}
```

```
    }  
}
```

**Output:**

10

## Types of Constructors

In Java, constructors can mainly be classified as:

### 1. Default Constructor

A **default constructor** is a constructor that takes **no arguments**. If no constructor is written by the programmer, Java compiler can provide a default constructor.

```
class Student {  
    Student() {  
        System.out.println("Default Constructor");  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
    }  
}
```

**Output:**

Default Constructor

---

### 2. Parameterized Constructor

A **parameterized constructor** is a constructor that accepts **one or more parameters** to initialize an object with specific values.

```
class Student {  
    int rollNo;  
    String name;  
  
    Student(int r, String n) {  
        rollNo = r;  
        name = n;  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student(101, "Rahul");  
  
        System.out.println(s.rollNo);  
        System.out.println(s.name);  
    }  
}
```

**Output:**

### 3. Copy Constructor

A **copy constructor** creates a new object by **copying the values of another object**.

Java does not provide a built-in copy constructor like C++, but we can create one using a constructor that accepts an object of the same class.

```
class Student {
    int rollNo;

    Student(int r) {
        rollNo = r;
    }

    Student(Student s) {
        rollNo = s.rollNo;
    }

    public static void main(String[] args) {
        Student s1 = new Student(101);
        Student s2 = new Student(s1);

        System.out.println(s2.rollNo);
    }
}
```

**Output:**

101

### Conclusion

A constructor is used to **initialize objects when they are created**. The commonly discussed types are **default constructor, parameterized constructor, and copy constructor**. Constructors make object initialization easier and organized.

### Q.2. Write a program to demonstrate copy constructor.

#### Definition

A **copy constructor** is a constructor that creates a new object by **copying the values of an existing object**.

In Java, a copy constructor is created by passing an object of the **same class** as a parameter.

#### Program

```
class Student {
    int rollNo;
    String name;

    // Parameterized constructor
    Student(int r, String n) {
        rollNo = r;
        name = n;
    }

    // Copy constructor
    Student(Student s) {
        rollNo = s.rollNo;
        name = s.name;
    }

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }

    public static void main(String[] args) {
        Student s1 = new Student(101, "Rahul");

        // Creating s2 by copying s1
        Student s2 = new Student(s1);

        s2.display();
    }
}
```

## Output

Roll No: 101

Name: Rahul

## Explanation

- s1 is the **original object**.
- Student(Student s) is the **copy constructor**.
- new Student(s1) creates a new object s2 by copying the values of s1.
- Therefore, s2 contains the same rollNo and name as s1.

## Key Point

**Copy constructor** = creates a new object by copying another object's data.

## Q.3. Explain method overloading with example.

### Definition

**Method overloading** means defining **more than one method with the same name but different parameters** in the same class.

It is an example of **compile-time polymorphism**.

### Characteristics

1. Method name must be the same.
2. Parameters must be different.
3. Parameters can differ in **number, type, or order**.
4. Return type alone cannot be used to overload a method.
5. It improves code readability and reusability.

### Example

```
class Calculator {  
  
    // Method with 2 parameters  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method with 3 parameters  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
}
```

```

// Method with different data type
double add(double a, double b) {
    return a + b;
}

public static void main(String[] args) {
    Calculator c = new Calculator();

    System.out.println(c.add(10, 20));
    System.out.println(c.add(10, 20, 30));
    System.out.println(c.add(10.5, 20.5));
}
}

```

## Output

```

30
60
31.0

```

## Explanation

Here, the `add()` method is defined **three times**:

- `add(int, int)` → accepts 2 integer values.
- `add(int, int, int)` → accepts 3 integer values.
- `add(double, double)` → accepts 2 double values.

All have the **same method name**, but their parameter lists are different. Therefore, this is called **method overloading**.

## Conclusion

Method overloading allows a class to have **multiple methods with the same name but different parameters** and provides **compile-time polymorphism**.

## Q.4. What is class? Explain how to create object of class and access rule of that.

### 1. Definition of Class

A **class** is a blueprint or template used to create objects. It contains **data members (variables)** and **member methods** that define the properties and behavior of objects.

## Syntax of Class

```
class ClassName {  
    // data members  
    // methods  
}
```

### Example

```
class Student {  
    int rollNo;  
    String name;  
  
    void display() {  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Name: " + name);  
    }  
}
```

Here, `Student` is a class containing two data members, `rollNo` and `name`, and one method, `display()`.

---

## 2. Creating an Object of a Class

An **object** is an instance of a class. In Java, an object is generally created using the `new` keyword.

### Syntax

```
ClassName objectName = new ClassName();
```

### Example

```
Student s1 = new Student();
```

Here:

- `Student` → class name
- `s1` → reference variable
- `new` → creates a new object
- `Student()` → calls the constructor

---

## 3. Accessing Members of a Class

The **dot (.) operator** is used to access the variables and methods of an object.

## Example

```
s1.rollNo = 101;  
s1.name = "Rahul";
```

```
s1.display();
```

Here, rollNo, name, and display( ) are accessed using the object s1.

---

## 4. Complete Program

```
class Student {  
    int rollNo;  
    String name;  
  
    void display() {  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Name: " + name);  
    }  
  
    public static void main(String[] args) {  
  
        // Creating object  
        Student s1 = new Student();  
  
        // Accessing data members  
        s1.rollNo = 101;  
        s1.name = "Rahul";  
  
        // Accessing method  
        s1.display();  
    }  
}
```

## Output

Roll No: 101

Name: Rahul

---

## 5. Access Rules of Class Members

Java provides **access modifiers** to control access to class members.

| Access Modifier | Access Rule                                               |
|-----------------|-----------------------------------------------------------|
| public          | Can be accessed from anywhere                             |
| private         | Can be accessed only within the same class                |
| protected       | Can be accessed within the same package and by subclasses |
| Default         | Can be accessed within the same package                   |

### Example of private

```
class Student {  
    private int rollNo;  
  
    void setRollNo(int r) {  
        rollNo = r;  
    }  
  
    void display() {  
        System.out.println(rollNo);  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
  
        s.setRollNo(101);  
        s.display();  
    }  
}
```

A **private** variable cannot be directly accessed outside its class. It is accessed through methods such as `setRollNo()`.

### Conclusion

A **class is a blueprint for creating objects**, while an **object is an instance of a class**. Objects are created using the **new** keyword, and class members are normally accessed using the **dot (.) operator**, subject to their access modifiers.

## Q.5. What is constructor? Explain types of constructor with example(any one).

### Definition

A **constructor** is a special member of a class that is used to **initialize an object**.

### Characteristics of Constructor

1. A constructor has the **same name as the class**.
2. It does **not have a return type**, not even `void`.
3. It is automatically called when an object is created.
4. A class can have **more than one constructor**.
5. Constructors can be **overloaded**.

---

## Types of Constructors

The main types of constructors are:

1. **Default Constructor**
2. **Parameterized Constructor**
3. **Copy Constructor**

---

### 1. Default Constructor

A **default constructor** is a constructor that has **no parameters**.

```
class Student {  
    int rollNo;  
  
    Student() {  
        rollNo = 101;  
    }  
  
    void display() {  
        System.out.println("Roll No: " + rollNo);  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.display();  
    }  
}
```

```
    }  
}
```

### Output:

Roll No: 101

---

## 2. Parameterized Constructor

A **parameterized constructor** accepts one or more parameters and is used to initialize an object with specific values.

```
class Student {  
    int rollNo;  
    String name;  
  
    Student(int r, String n) {  
        rollNo = r;  
        name = n;  
    }  
  
    void display() {  
        System.out.println("Roll No: " + rollNo);  
        System.out.println("Name: " + name);  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student(101, "Rahul");  
        s.display();  
    }  
}
```

### Output:

Roll No: 101

Name: Rahul

---

## 3. Copy Constructor

A **copy constructor** creates a new object by copying the values of another object of the same class.

```
Student(Student s) {  
    rollNo = s.rollNo;  
    name = s.name;  
}
```

Java does not provide a special built-in copy-constructor feature like C++, but a programmer can define one manually.

## Conclusion

A constructor is mainly used for **object initialization**. The commonly discussed types are **default, parameterized, and copy constructors**. For the exam, one type with a suitable program is enough when the question specifically says “**any one.**”

## Q.6. Explain method overriding with suitable example.

### Definition

**Method overriding** is a feature of Java in which a **subclass provides its own implementation of a method that is already defined in its superclass**.

It is used to achieve **runtime polymorphism**.

### Important Points

1. Method overriding requires **inheritance**.
2. The method name must be the **same**.
3. The parameters must be the **same**.
4. The subclass provides a different implementation of the inherited method.
5. It occurs between a **superclass and subclass**.
6. It supports **runtime polymorphism**.

---

## Example

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {
```

```
// Overriding sound() method
void sound() {
    System.out.println("Dog barks");
}

public static void main(String[] args) {

    Dog d = new Dog();
    d.sound();
}
}
```

## Output

Dog barks

## Explanation

Here, the `Animal` class contains the `sound()` method.

The `Dog` class inherits the `sound()` method from `Animal` and provides its **own implementation**:

```
void sound() {
    System.out.println("Dog barks");
}
```

Therefore, the `sound()` method in `Dog` **overrides** the `sound()` method of `Animal`.

## Conclusion

**Method overriding allows a subclass to provide a specific implementation of a method inherited from its superclass. It is an important feature of runtime polymorphism in Java.**

## Q.7. Explain **this** keyword with suitable examples.

### Definition

The **this** keyword in Java is a reference variable that refers to the **current object of a class**.

It is mainly used when instance variables and local variables have the same name. Java, in its infinite wisdom, lets you name both things `name`, then gives you `this.name` to sort out the mess. 😊

### Uses of **this** Keyword

1. To refer to the **current class instance variable**.

2. To call the **current class method**.
  3. To call another **constructor of the same class**.
  4. To pass the current object as an argument.
- 

## 1. To Refer to Current Instance Variable

```
class Student {  
    int rollNo;  
    String name;  
  
    Student(int rollNo, String name) {  
        this.rollNo = rollNo;  
        this.name = name;  
    }  
  
    void display() {  
        System.out.println("Roll No: " + this.rollNo);  
        System.out.println("Name: " + this.name);  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student(101, "Rahul");  
        s.display();  
    }  
}
```

### Output

Roll No: 101

Name: Rahul

### Explanation

Here, `rollNo` and `name` are constructor parameters, while `this.rollNo` and `this.name` refer to the **instance variables of the current object**.

---

## 2. To Call Current Class Method

The `this` keyword can also be used to call a method of the current class.

```
class Student {  
  
    void display() {  
        System.out.println("Hello Student");  
    }  
  
    void show() {  
        this.display();  
    }  
  
    public static void main(String[] args) {  
        Student s = new Student();  
        s.show();  
    }  
}
```

## Output

Hello Student

Here, `this.display()` calls the `display()` method of the current object.

---

## 3. To Call Another Constructor

The `this()` statement can be used to call another constructor of the same class.

```
class Student {  
  
    Student() {  
        this(101);  
        System.out.println("Default Constructor");  
    }  
  
    Student(int rollNo) {  
        System.out.println("Roll No: " + rollNo);  
    }  
  
    public static void main(String[] args) {
```

```
        Student s = new Student();
    }
}
```

## Output

Roll No: 101

Default Constructor

## Important Points

- `this` refers to the **current object**.
- `this()` is used to call another constructor of the same class.
- `this` cannot be used in a **static context** to refer to a current object.
- It helps distinguish between **instance variables and local variables**.

## Conclusion

The **this** keyword represents the **current object** and is commonly used to access instance variables, call methods, and invoke another constructor of the same class.

## Q.8. Explain constructor overloading with suitable Java program.

### Definition

**Constructor overloading** means defining **more than one constructor in the same class with different parameter lists**.

It allows objects of the same class to be initialized in **different ways**.

### Important Points

1. All constructors have the **same name as the class**.
2. Constructors must have **different parameter lists**.
3. The difference can be in the **number or type of parameters**.
4. Constructor overloading is an example of **compile-time polymorphism**.
5. It provides flexibility in object initialization.

---

## Java Program

```
class Student {
    int rollNo;
    String name;

    // Constructor 1: No parameters
```

```
Student() {  
    rollNo = 0;  
    name = "Unknown";  
}
```

// Constructor 2: One parameter

```
Student(int r) {  
    rollNo = r;  
    name = "Unknown";  
}
```

// Constructor 3: Two parameters

```
Student(int r, String n) {  
    rollNo = r;  
    name = n;  
}
```

```
void display() {  
    System.out.println("Roll No: " + rollNo);  
    System.out.println("Name: " + name);  
    System.out.println();  
}
```

```
public static void main(String[] args) {
```

```
    Student s1 = new Student();  
    Student s2 = new Student(101);  
    Student s3 = new Student(102, "Rahul");
```

```
    s1.display();  
    s2.display();  
    s3.display();
```

```
}
```

```
}
```

## Output

Roll No: 0

Name: Unknown

Roll No: 101

Name: Unknown

Roll No: 102

Name: Rahul

## Explanation

The Student class contains **three constructors**:

- `Student()` → takes no parameter.
- `Student(int r)` → takes one parameter.
- `Student(int r, String n)` → takes two parameters.

When an object is created, Java selects the appropriate constructor according to the **arguments passed**.

For example:

```
Student s1 = new Student();
```

```
Student s2 = new Student(101);
```

```
Student s3 = new Student(102, "Rahul");
```

Therefore, this is called **constructor overloading**.

## Conclusion

Constructor overloading allows a class to have **multiple constructors with different parameters**, making it possible to initialize objects in different ways.

## Q.9. Explain static variable and static method with suitable examples.

### 1. Static Variable

A **static variable** is a variable declared using the `static` keyword. It belongs to the **class rather than individual objects**.

Only **one copy** of a static variable is created, and it is shared by all objects of the class.

### Example of Static Variable

```
class Student {
```

```
int rollNo;

static String college = "ABC College";

Student(int r) {
    rollNo = r;
}

void display() {
    System.out.println("Roll No: " + rollNo);
    System.out.println("College: " + college);
}

public static void main(String[] args) {
    Student s1 = new Student(101);
    Student s2 = new Student(102);

    s1.display();
    s2.display();
}
}
```

## Output

```
Roll No: 101
College: ABC College
Roll No: 102
College: ABC College
```

## Explanation

Here, college is a **static variable**. It is shared by both s1 and s2.

If the value of college is changed, the changed value will be available to all objects because there is only **one shared copy**.

---

## 2. Static Method

A **static method** is a method declared using the `static` keyword. It belongs to the **class** and can be called without creating an object.

### Example of Static Method

```
class Calculator {  
  
    static int square(int n) {  
        return n * n;  
    }  
  
    public static void main(String[] args) {  
  
        System.out.println("Square = " + Calculator.square(5));  
    }  
}
```

### Output

Square = 25

### Explanation

Here, `square( )` is a **static method**. It is called using the class name:

`Calculator.square(5);`

There is no need to create an object of `Calculator`.

---

## Important Points of Static Variable

1. Declared using the `static` keyword.
2. It belongs to the class.
3. Only one copy is created.
4. It is shared by all objects.
5. It can be accessed using the class name.

Example:

`Student.college`

---

## Important Points of Static Method

1. Declared using the `static` keyword.
2. It belongs to the class.
3. It can be called without creating an object.
4. It can directly access static members.
5. The `main()` method in Java is static.

### Difference

| Static Variable                             | Static Method                             |
|---------------------------------------------|-------------------------------------------|
| Stores shared data.                         | Performs an operation.                    |
| Declared using <code>static</code> .        | Declared using <code>static</code> .      |
| Shared among all objects.                   | Belongs to the class.                     |
| Example: <code>static String college</code> | Example: <code>static int square()</code> |

### Conclusion

A **static variable** is used to store data shared by all objects of a class, while a **static method** is used to perform an operation that belongs to the class and can be called without creating an object. 📄

## Q.10. Differentiate between method overloading and method overriding with examples.

### Definition

#### Method Overloading:

Method overloading means defining **multiple methods with the same name but different parameters** in the same class.

#### Method Overriding:

Method overriding means defining a method in a subclass with the **same name and same parameters** as a method in its superclass, but with a different implementation.

---

## Difference Between Method Overloading and Method Overriding

| Method Overloading                                     | Method Overriding                                  |
|--------------------------------------------------------|----------------------------------------------------|
| 1. Same method name with <b>different parameters</b> . | 1. Same method name with <b>same parameters</b> .  |
| 2. Usually occurs within the <b>same class</b> .       | 2. Occurs between <b>superclass and subclass</b> . |

| Method Overloading                                            | Method Overriding                                            |
|---------------------------------------------------------------|--------------------------------------------------------------|
| 3. <b>Inheritance is not required.</b>                        | 3. <b>Inheritance is required.</b>                           |
| 4. It is <b>compile-time polymorphism</b> .                   | 4. It is <b>runtime polymorphism</b> .                       |
| 5. Parameters must be different.                              | 5. Parameters must be the same.                              |
| 6. Return type alone cannot differentiate overloaded methods. | 6. The overriding method must have a compatible return type. |
| 7. It provides multiple ways to perform a similar operation.  | 7. It allows a subclass to provide its own implementation.   |

## Example of Method Overloading

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }

    public static void main(String[] args) {
        Calculator c = new Calculator();

        System.out.println(c.add(10, 20));
        System.out.println(c.add(10, 20, 30));
    }
}
```

## Output

```
30
60
```

## Explanation

Here, both methods have the name `add()`, but their parameter lists are different:

```
add(int, int)
```

```
add(int, int, int)
```

Therefore, it is **method overloading**.

---

## Example of Method Overriding

```
class Animal {  
  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}
```

```
class Dog extends Animal {  
  
    void sound() {  
        System.out.println("Dog barks");  
    }  
  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.sound();  
    }  
}
```

### Output

Dog barks

### Explanation

The `Animal` class has a `sound()` method. The `Dog` class inherits it and provides its **own implementation** of `sound()`.

Therefore, it is **method overriding**.

---

### Conclusion

Method overloading allows multiple methods with the same name but different parameters, whereas method overriding allows a subclass to provide a different implementation of a method inherited from its superclass.

## Q.11. Explain default constructor and parameterized constructor with examples.

### 1. Default Constructor

A **default constructor** is a constructor that **does not take any parameters**. It is used to initialize an object with default or predefined values.

#### Example

```
class Student {
    int rollNo;
    String name;

    // Default constructor
    Student() {
        rollNo = 101;
        name = "Rahul";
    }

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }

    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

#### Output

```
Roll No: 101
Name: Rahul
```

#### Explanation

Here, `Student ( )` is a **default constructor** because it does not accept any parameters.

When the statement:

```
Student s = new Student();
```

is executed, the constructor is automatically called and initializes the object.

**Important:** If a class has **no constructor written by the programmer**, Java provides a compiler-generated default constructor. This is different from a user-defined no-argument constructor like the one above.

---

## 2. Parameterized Constructor

A **parameterized constructor** is a constructor that accepts **one or more parameters**. It is used to initialize an object with specific values.

### Example

```
class Student {
    int rollNo;
    String name;

    // Parameterized constructor
    Student(int r, String n) {
        rollNo = r;
        name = n;
    }

    void display() {
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
    }

    public static void main(String[] args) {
        Student s = new Student(102, "Amit");
        s.display();
    }
}
```

### Output

```
Roll No: 102
Name: Amit
```

### Explanation

Here, `Student(int r, String n)` is a **parameterized constructor** because it accepts two parameters.

The values `102` and `"Amit"` are passed while creating the object:

```
Student s = new Student(102, "Amit");
```

These values are assigned to `rollNo` and `name`.

---

## Difference Between Default and Parameterized Constructor

| Default Constructor                                   | Parameterized Constructor                       |
|-------------------------------------------------------|-------------------------------------------------|
| Does not take parameters.                             | Takes one or more parameters.                   |
| Initializes an object with default/predefined values. | Initializes an object with specific values.     |
| Example: <code>Student( )</code>                      | Example: <code>Student(int r, String n)</code>  |
| Object can be created without passing arguments.      | Arguments are passed while creating the object. |

### Conclusion

A **default constructor** initializes an object without receiving parameters, while a **parameterized constructor** initializes an object using values supplied during object creation. Both are useful for initializing objects in Java. 